GRAPH TRAVERSAL (DFS)

GROUP 5

Start Slide

Meet Our Team

NAVANEETH K SANU - KH.EN.U4BCA24047 (TEAM LEAD)

ACHYUTH N - KH.EN.U4BCA24004

ADITHYA RANJITH - KH.EN.U4BCA24005

GOWTHAMI P V - KH.EN.U4BCA24029

HARISREERAM RAJESH - KH.EN.U4BCA24032

LIYAN SEBASTIAN - KH.EN.U4BCA24041

NIRANJANA T J - KH.EN.U4BCA24051

GRAPH TRAVERSAL

- A graph is made up of nodes connected by edges.
- To explore or visit all the nodes in a graph, we use a process called graph traversal.
- Graph traversal means visiting each node of a graph in a particular order so that no node is left unvisited.
- It helps us understand the structure of the graph and how different nodes are connected to each other.
- Traversal is important in many areas such as path finding, network analysis, and data searching.
- By using traversal, we can explore the graph completely and collect useful information from it.
- There are two main types of graph traversal:

    Depth First Search (DFS)

    Breadth First Search (BFS)

1

2

3

DEPTH FIRST SEARCH

- In Depth First Search (or DFS) for a graph, we traverse all adjacent vertices one by one.
- When we traverse an adjacent vertex, we completely finish the traversal of all vertices reachable through that adjacent vertex.
- This is similar to a depth first tree traversal, where we first completely traverse the left subtree and then move to the right subtree.
- The key difference is that, unlike trees, graphs may contain cycles (a node may be visited more than once). To avoid processing a node multiple times, we use a boolean visited array.

2

3

4

DEPTH FIRST SEARCH

The source vertex s is 0. We visit it first, then we visit an adjacent.

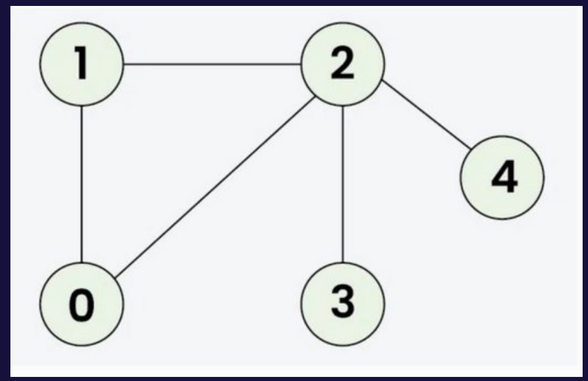
Start at 0: Mark as visited. Output: 0

Move to 1: Mark as visited. Output: 1

Move to 2: Mark as visited. Output: 2

Move to 3: Mark as visited. Output: 3 (backtrack to 2)

Move to 4: Mark as visited. Output: 4 (backtrack to 2, then backtrack to 1, then to 0)



Note: There can be multiple DFS traversals of a graph according to the order in which we pick adjacent vertices. Here we pick vertices as per the insertion order.

ALGORITHM

STEP 1: Start from any node.

STEP 2: Mark the node as visited.

STEP 3: Visit one unvisited neighbor of that node.

STEP 4: Repeat steps 2 and 3 until there are no unvisited neighbors.

STEP 5: When stuck, go back (backtrack) to the previous node.

STEP 6: Continue the process until all nodes are visited.

WORKING OF DFS

Consider the graph:



Start from A, mark visited → Output: A

Go to B, mark visited → Output: A, B

Go to D, mark visited → Output: A, B, D

No unvisited adjacent vertex of D → Backtrack to B

Next unvisited adjacent vertex of B → E, mark visited → Output: A, B, D, E

Backtrack to A → Next adjacent vertex of A → C, mark visited → Output: A, B, D, E, C

Final DFS traversal: A → B → D → E → C

5

6

4

STACK IN DFS

- DFS uses a Stack to keep track of the nodes during traversal.
- Stack follows **LIFO** (Last In, First Out) order.
- When DFS visits a node, it **pushes** that node into the stack.
- When no unvisited neighbor is found, it **pops** the top node to backtrack.
- Stack keeps track of all the nodes that still have unvisited neighbors.
- DFS can use either:

    System Stack

    User-defined Stack

- Without a stack, DFS cannot remember which node to return to.

IMPORTANCE OF STACK

- **Helps in Backtracking :** The main purpose of the stack in DFS is to go back (backtrack) when no further nodes can be visited.
- **Stores Visited Path :** The stack remembers the path that has been followed so far.
- **Supports Depth-wise Traversal :** Because of its LIFO (Last In, First Out) nature, the stack lets DFS go deep into one path before exploring another.
- **Maintains Order of Exploration :** The stack ensures nodes are visited in the correct order.
- **Essential for Functioning of DFS :** Without a stack, DFS cannot remember where it came from, so it wouldn’t be able to complete the traversal correctly

PROPERTIES OF DFS

1.**Data Structure:** It is fundamentally based on a Stack (LIFO - Last-In, First-Out). This is either an explicit stack data structure or the implicit "call stack" used by recursion.

2.**Exploration Strategy:** DFS explores deeply. It follows a single path as far as possible until it reaches a dead end.

3.**Backtracking:** When it hits a dead end (a node with no unvisited neighbors), it backtracks to the previous node to explore its next available path.

4.**Completeness:** DFS is complete. If a node is reachable from the start node, DFS is guaranteed to find it (assuming a finite graph).

PROPERTIES OF DFS

5.**Time Complexity:** O(V + E), where V is the number of vertices (nodes) and E is the number of edges. This is because it visits every vertex and checks every edge once.

6.**Space Complexity:** O(V) or O(H) (worst case), where H is the maximum height/depth of the graph. The space is determined by the maximum size of the stack.

7.**Connectivity:** It is highly effective at finding connected components in a graph.

ADVANTAGES OF DFS

- **Memory efficient**: DFS uses less memory than Breadth-First Search (BFS) because it only needs to store the nodes along the current path, not all nodes at a given level.
- **Simple implementation**: The recursive nature of DFS makes it easy to write and understand.
- **Faster for certain problems**: If a solution exists deep in the graph, DFS can find it faster than BFS by not exploring all nodes at the same level first.
- **Useful for pathfinding and cycle detection**: DFS is excellent for tasks like finding a path between two nodes, checking if a graph is connected, and detecting if a graph contains a cycle.

LIMITATIONS OF DFS

- **Not guaranteed to find the shortest path**: DFS can find a longer path to the goal before finding the shortest one, especially in unweighted graphs.
- **Incomplete without a depth bound**: Without a depth limit, DFS might not find a solution even if one exists, as it could get stuck exploring an infinitely deep path.
- **Can get trapped in cycles**: If a graph contains cycles, DFS can potentially run forever. A mechanism to track visited nodes is necessary to avoid this infinite loop.
- **Potentially inefficient**: If the graph is very large and interconnected, DFS might explore a significant portion of the graph before finding the solution, making it slower than other algorithms in some cases.

APPLICATIONS OF DFS

1. Path Finding: DFS helps find a path between two nodes in a graph

Useful in: Navigation systems ,Maze solving, Game AI (finding routes or escape paths)

2. Cycle Detection: DFS can detect cycles in both directed and undirected graphs.

In undirected graphs, if a visited vertex is reached again (not the parent), there's a cycle.

In directed graphs, if we visit a node that is still in the recursion stack, a cycle exists.

Used in: Deadlock detection, dependency resolution.

3. Topological Sorting: DFS is used to perform topological sorting on a Directed Acyclic Graph (DAG).It arranges nodes in an order such that all dependencies come before dependent nodes.

Used in: Task scheduling, compiler dependency checking.

APPLICATIONS OF DFS

4. Connected Components: DFS helps identify connected components in a graph — sets of nodes that are reachable from each other.

Used in: Social network analysis, cluster detection.

5.Solving Mazes & Puzzles: DFS explores all possible paths deeply before backtracking — ideal for: Maze traversal,Sudoku and N-Queens problems

Used in: Game solving algorithms.

6.Strongly Connected Components (SCC): Algorithms like Kosaraju's and Tarjan's use DFS to find SCCs in directed graphs.

Used in: Web link analysis (like Google PageRank), circuit design.

7. Tree Traversals: In trees (special type of graph), preorder, inorder, and postorder traversals are all forms of DFS.

Used in: Expression tree evaluation, syntax parsing.

Thank You

FOR YOUR ATTENTION